Softwareprojekt

GPX-Visualizer und CampusKompass

Vereinfachte Code- und Anforderungspräsentation

Präsentationsstruktur

1. Ziel und Funktionen
2. Architektur
3. Modellierung
4. Bedienung und Weboberfläche
5. Tests und Erweiterbarkeit

1. Ziel und Funktionen

Ziel

CampusKompass bündelt die wichtigsten Informationen für einen Studientag an den Standorten Schmalkalden und Gera in einer lokalen Weboberfläche. Die Anwendung reduziert den Wechsel zwischen mehreren externen Informationsquellen und ordnet die Daten dem ausgewählten Datum zu.

Funktionen

- Studienplan nach Datum filtern und Termine mit Zeit, Veranstaltung, Lehrkraft, Ort und Raum anzeigen
- Mensaplan des passenden Campus über den lokalen Server laden
- Wetter für Campus, Datum und Uhrzeit anzeigen
- Campusabhängiges Ziel für Route und ÖPNV-Fahrplanauskunft setzen

Projekt auf einen Blick

- **GPX-Visualizer**: Wanderungen aus GPX-Dateien auswerten
- **CampusKompass**: Studienplan, Mensa, Wetter und Campusroute bündeln
- Zwei unabhängige Anwendungen in einem Repository
- Python für Analyse und Server
- Matplotlib bzw. HTML/CSS/JavaScript für die Darstellung

GPX-Visualizer: Datenfluss

```
Wanderung.gpx
      |
      v
GpxTrack.from_gpx()
      |
      v
GeoPoint -> Route -> Statistikberechnung
      |
      v
2D-Karte + Höhenprofil + Slider + 3D-Ansicht
```

GPX-Visualizer: wichtige Bausteine

- `GeoPoint` : GPS-Koordinate, Höhe und Zeitstempel
- `Route` : Sammlung der Trackpunkte
- `GpxTrack` : liest die GPX-XML-Datei
- `haversine_m` : berechnet Entfernungen
- `compute_statistics` : berechnet Strecke, Dauer, Höhenmeter und Tempo
- Matplotlib: Diagramme und Interaktion
- Contextily: Kartenhintergrund

GPX-Visualizer: Benutzeroberfläche

- Route mit Start- und Zielmarkierung
- Karten-Overlay in Web Mercator
- Gesamtstatistiken im Kartenbereich
- Höhenprofil über die Entfernung
- Slider verschiebt den aktuellen Positionsmarker
- Umschaltung zwischen 2D- und 3D-Ansicht

2. Architektur

Systemarchitektur

```
Browser
  |
  +--> index.html / JavaScript
  |       +--> study-plan.json
  |       +--> Open-Meteo
  |       +--> Google Maps
  |       +--> /api/mensa
  |
  +--> server.py
          +--> statische Dateien
          +--> Proxy zum Studierendenwerk Thüringen
```

CampusKompass: Ablauf

1. Nutzer wählt ein Datum.
2. Der Studienplan wird nach diesem Datum gefiltert.
3. Der Studienort bestimmt Mensa, Wetterstandort und Routenziel.
4. Mensaplan und Wetter werden geladen.
5. Die Campusroute wird als Wegbeschreibung angezeigt.

Der Proxy in `server.py` verhindert, dass der Browser den Mensa-Endpunkt direkt wegen CORS ansprechen muss.

3. Modellierung: Klassendiagramm Backend

Das Backend besteht aus dem Modul `server.py` und dem spezialisierten Handler `DashboardHandler`:

```
server.py
  +-- main()
  +-- ThreadingHTTPServer
          +-- DashboardHandler
                  +-- do_GET()
                  +-- /api/mensa -> Studierendenwerk Thüringen
```

`DashboardHandler` erweitert `SimpleHTTPRequestHandler`. Dadurch werden statische Dateien ausgeliefert; der zusätzliche API-Pfad validiert Datum und Mensa-ID und leitet die Anfrage als POST weiter.

Quelle: `modellierung/klassendiagramm_backend.puml`

Modellierung: Klassendiagramm Frontend

Das JavaScript in `index.html` ist prozedural organisiert. Für die Modellierung wird es als `CampusDashboard` mit seinen Zuständigkeiten dargestellt:

```
CampusDashboard
  +-- StudySession[] aus study-plan.json
  +-- loadStudyPlan() / renderStudyPlan()
  +-- loadMenu() -> Meal[]
  +-- loadWeather() -> WeatherData
  +-- showRoute() -> RouteDestination
```

Die Funktionen wählen anhand des Datums Mensa, Wetterstandort und Routenziel aus und aktualisieren anschließend die Oberfläche.

Quelle: `modellierung/klassendiagramm_frontend.puml`

Modellierung: Paketdiagramm

```
CampusKompass
  +-- Frontend: index.html
  +-- Backend: server.py
  +-- Daten: study-plan.json, assets/

Frontend -> Backend -> Studierendenwerk Thüringen
Frontend -> Open-Meteo
Frontend -> Google Maps
Frontend -> VMT Fahrplanauskunft
```

Die Paketgrenzen machen sichtbar, dass nur der Mensa-Aufruf über das Backend läuft. Wetter, Karten, Fahrplan und Studienplandaten werden vom Frontend verarbeitet.

Quelle: `modellierung/paketdiagramm.puml`

4. Bedienung und Weboberfläche

Bedienablauf

1. `python .\server.py` starten und `http://127.0.0.1:8000/` öffnen.
2. Datum und Abfahrtszeit im Bereich „Tagesplanung“ auswählen.
3. Studienplan, Mensa und Wetter für die Auswahl prüfen.
4. Im Fahrplan-Widget Startort und das angezeigte Campusziel verwenden.
5. Für eine Autofahrt Startpunkt eingeben oder den Standort freigeben.
6. Mit „Jetzt“ jederzeit zur aktuellen Zeit zurückkehren.

Oberfläche

- Kopfbereich mit Hochschullinks und aktuellem Datum/Uhrzeit
- Tagesplanung als zentrale Eingabe für Datum und Zeit
- Karten für Wetter, Mensa, Studienplan, Fahrplan und Route

5. Tests und Erweiterbarkeit

Durchgeführte Prüfungen

- Python-Syntaxprüfung mit `python -m py_compile .\server.py`
- JavaScript-Syntaxprüfung mit `node --check`
- JSON-Validierung von `study-plan.json`
- HTTP-Smoke-Test des lokalen Servers
- Prüfung ungültiger Mensa-IDs mit erwarteter HTTP-400-Antwort
- VS-Code-Diagnose ohne gemeldete Fehler oder Warnungen
- Manuelle Prüfung von Datumswechsel, Campuswechsel, Wetterfehlern und Standortfreigabe

Die externen Dienste bleiben abhängig von Netzwerk, Verfügbarkeit und ihrer aktuellen Antwortstruktur. Das Testkonzept ist in

MoSCoW-Methode

- **Must have**: unverzichtbar für das MVP
- **Should have**: wichtig, aber verschiebbar
- **Could have**: nützliche Erweiterung
- **Won't have**: in dieser Version bewusst ausgeschlossen

Die vollständige Liste steht in `requirements/userstories-moscow.md`.

Die User Stories sind über IDs mit den Use Cases in `requirements/usecases.md` rückverfolgbar.

Priorisierte GPX-User-Stories

Must have

- GPX-Datei importieren
- Route in 2D anzeigen
- Strecke, Dauer, Höhenmeter und Geschwindigkeit anzeigen

Should have

- Höhenprofil anzeigen
- Karten-Overlay anzeigen
- Position über Slider verfolgen

Could have

- Interaktive 3D-Ansicht

Priorisierte CampusKompass-Stories

Must have

- Studienplan für ein Datum anzeigen

Should have

- Passenden Mensaplan anzeigen
- Wetter am Campus anzeigen

Could have

- Route zum Campus planen
- Fahrplanauskunft für Datum und Uhrzeit öffnen
- Dashboard-Auswahl auf „Jetzt“ setzen

Use Cases: CampusKompass

ID	Use Case	Hauptakteur
UC-01	Dashboard öffnen	Student/in
UC-02	Studienplan für ein Datum anzeigen	Student/in
UC-03	Mensaplan laden	Student/in
UC-04	Wetterdaten anzeigen	Student/in
UC-05	Route zum Campus planen	Student/in
UC-06	Fahrplan für Datum und Uhrzeit öffnen	Student/in
UC-07	Auswahl auf aktuelle Zeit setzen	Student/in

Die Use Cases beschreiben zusätzlich Vorbedingungen, Alternativfälle, externe Dienste und die erwarteten Ergebnisse. Die vollständige Spezifikation

Rückverfolgbarkeit der Anforderungen

```
User Story CAMPUS-01 -> UC-02 -> study-plan.json
User Story CAMPUS-02 -> UC-03 -> server.py -> Studierendenwerk
User Story CAMPUS-03 -> UC-04 -> Open-Meteo
User Story CAMPUS-04 -> UC-05 -> Google Maps
User Story CAMPUS-05 -> UC-06 -> VMT
User Story CAMPUS-06 -> UC-07 -> Datums-/Zeitsteuerung
```

Die Anforderungen, Use Cases und Modellierungsdiagramme bilden damit eine durchgängige Dokumentationskette vom Nutzerziel bis zur technischen Struktur.

Qualitäts- und Umsetzungsstand

- GPX-Import und zentrale Visualisierung sind vorhanden.
- Statistiken werden aktuell direkt in `src/main.py` berechnet.
- Einige Modellmethoden in `src/model/analysis.py` sind noch vorbereitet, aber nicht vollständig implementiert.
- CampusKompass wird mit `python .\campuskompass\server.py` gestartet.
- Die Anwendungen benötigen für externe Karten-, Wetter- und Mensadaten eine Netzwerkverbindung.

Start der Anwendungen

GPX-Visualizer

```
python .\src\main.py
```

CampusKompass

```
cd ".\campuskompass"
python .\server.py
```

Danach: http://127.0.0.1:8000/

Fazit

- Das Projekt verbindet Datenmodell, Analyse und Visualisierung.
- Der GPX-Visualizer unterstützt die technische Routenauswertung.
- CampusKompass bündelt studienbezogene Informationen in einer Weboberfläche.
- Die MoSCoW-Priorisierung trennt MVP-Funktionen von Erweiterungen.
- Nächster sinnvoller Ausbau: mehrere GPX-Dateien und vollständige Modell-Analysefunktionen.